

UE : Logique

Arithmétique binaire et ternaire en Prolog

Étudiantes :

Lylia IMLOUL
Lyna HADJI

Encadrant :

Tristan VACCON

Table des matières

1. **Introduction** 3
2. **Représentations et structures de données** 3
3. **Partie 1 – Arithmétique binaire (modulo 2)** 3
4. **Partie 2 – Arithmétique ternaire (modulo 3)** 7
5. **Les 3M** 9
6. **Conclusion** 11
7. **Bibliographie** 11

1. Introduction

Ce projet a pour objectif de manipuler des vecteurs et des matrices en Prolog dans deux contextes d'arithmétique modulaire:

- Arithmétique binaire (modulo 2) utilisant les symboles **o** (0) et **u** (1).
- Arithmétique ternaire (modulo 3) utilisant les symboles **o** (0), **u** (1) et **d** (2).

Le but est de trouver des vecteurs V vérifiant :

$$V \times M = Y$$

Où M est une matrice donnée et Y un vecteur résultat, avec des contraintes sur le nombre de coefficients non nuls dans V.

Ce projet permet de mettre en pratique :

- La programmation logique en Prolog
- La récursivité
- La manipulation de listes
- La gestion de contraintes

2. Représentations et structures de données

2.1. Valeurs

Nous utilisons **o** pour 0 et **u** pour 1 en binaire, et nous ajoutons **d** pour 2 en ternaire. Ces valeurs sont déclarées par des faits Prolog :

```
valeur_binaire(o).
valeur_binaire(u).
```

```
valeur_ternaire(o).
valeur_ternaire(u).
valeur_ternaire(d).
```

2.2. Vecteurs et matrices

- **Vecteurs :** Représentés par des listes simples, par exemple [u, o, u, u].
- **Matrices :** Représentés par des listes de listes, où chaque sous-liste est une ligne, par exemple M=[[u, o], [o, u],[u, u],[o, u]].

3. Partie 1 – Arithmétique binaire (modulo 2)

3.1. Opérations de base : add/3 et mul/3

L'addition et la multiplication modulo 2 sont définies par des tables de vérité :

```
add(o,o,o). add(o,u,u).
add(u,o,u). add(u,u,o).
```

```
mul(o,o,o). mul(o,u,o).
mul(u,o,o). mul(u,u,u).
```

Résultat :

```
| ?- add(u,u,X).

X = o

yes
| ?- mul(u,u,X).

X = u

yes
```

3.2. Addition de vecteurs : add_vect/3

L'addition terme à terme de deux vecteurs binaires est définie récursivement :

```
add_vect([],[],[]). % si les listes sont vides = résultat vide

add_vect([H1|T1],[H2|T2],[R|T]) :- % cas récursif
   add(H1,H2,R),
   add_vect(T1,T2,T).
```

- Cas de base : gère les listes vides (le résultat est également vide).
- Cas récursif : additionne les têtes via add/3, puis rappelle add_vect/3 sur les queues.

Résultat :

```
| ?- add_vect([u,o],[u,u],R).

R = [o,u]

yes
```

3.3. Produit vecteur-matrice : produit/3

On implémente V x M par récursion sur les éléments de V et les lignes de M simultanément :

```
produit([], _, []).

produit([o|T], [_|M], R) :-
   produit(T, M, R).

produit([u|T], [Ligne|M], R) :-
   produit(T, M, R1),
   ( R1 = [] -> R = Ligne ; add_vect(Ligne, R1, R) ).
```

Trois cas sont distingués :

- Cas de base : le vecteur V est vide, le résultat est [].
- Coefficient o : on ignore la ligne correspondante (0 x ligne = vecteur, neutre pour l'addition) et on continue la récursion.

- Coefficient u : on calcule d'abord le reste R1, puis on ajoute la ligne courante à R1. La garde R1=[] gère le cas particulier où R1 est encore vide (aucun u rencontré dans le reste du vecteur) : dans ce cas on retourne directement Ligne sans appeler add_vect.

Résultat :

```
| ?- M=[[u,o],[o,u],[u,u],[o,u]],produit([o,o,u,u],M,R).

M = [[u,o],[o,u],[u,u],[o,u]]
R = [u,o] ? ;

(31 ms) no
```

3.4. Génération des vecteurs : vecteur/2

```
vecteur([],0).

vecteur([H|T],N) :-
   N > 0,
   valeur_binaire(H),
   N1 is N-1,
   vecteur(T,N1).
```

Génère tous les vecteurs possibles de taille N (2^N).

Résultat :

```
| ?- vecteur(V,2).

V = [o,o] ? ;

V = [o,u] ? ;

V = [u,o] ? ;

V = [u,u] ? ;

no
```

3.5. Test du vecteur nul : zeros/1

La clause zeros/1 réussit si et seulement si tous les éléments d'une liste sont des o :

```
zeros([]).
zeros([o|T]) :-
   zeros(T).
```

Elle est utilisée dans solution/2 pour vérifier que le résultat du produit est bien nul.

3.6. Comptage des u : count_u/2

```
count_u([],0).

count_u([u|T],N) :-
   count_u(T,N1),
   N is N1 + 1.

count_u([o|T],N) :-
   count_u(T,N).
```

count_u/2 parcourt le vecteur et compte le nombre de u. Les o sont ignorés. Cette clause est utilisée dans solution_binaire pour filtrer les vecteurs selon le nombre de coefficient à 1.

Résultat :

```
| ?- count_u([u,o,u,u],N).

N = 3 ? ;

(16 ms) no
```

3.7. Clauses principales

solution/2 : trouve tous les V tels que V x M = vecteur nul.

```
solution(M,V) :-
   length(M,N),
   vecteur(V,N),
   produit(V,M,R),
   zeros(R).
```

Résultat :

```
| ?- M=[[u,o],[o,u],[u,u],[o,u]], solution(M,V).

M = [[u,o],[o,u],[u,u],[o,u]]
V = [o,o,o,o] ? ;

M = [[u,o],[o,u],[u,u],[o,u]]
V = [o,u,o,u] ? ;

M = [[u,o],[o,u],[u,u],[o,u]]
V = [u,o,u,u] ? ;

M = [[u,o],[o,u],[u,u],[o,u]]
V = [u,u,u,o] ? ;

(47 ms) no
```

solution_binaire/4 : version générale ; trouve V tel que V x M = Y avec exactement nombre coefficients u dans V.

```
solution_binaire(M,V,Y,Nombre) :-
   length(M,N),
   vecteur(V,N),
   count_u(V,Nombre),
   produit(V,M,Y).
```

Résultat :

```
| ?- M=[[u,o],[o,u],[u,u],[o,u]], solution_binaire(M,V,[o,o],2).

M = [[u,o],[o,u],[u,u],[o,u]]
V = [o,u,o,u] ? ;

no
```

4. Partie 2 – Arithmétique ternaire (modulo 3)

4.1. Opérations de base : add3/3 et mul3/3

Les tables d'addition et de multiplication modulo 3 couvrent les 9 combinaisons ordonnées pour chaque opération :

```
add3(o,o,o). add3(o,u,u). add3(o,d,d).
add3(u,o,u). add3(u,u,d). add3(u,d,o).
add3(d,o,d). add3(d,u,o). add3(d,d,u).

mul3(o,o,o). mul3(o,u,o). mul3(o,d,o).
mul3(u,o,o). mul3(u,u,u). mul3(u,d,d).
mul3(d,o,o). mul3(d,u,d). mul3(d,d,u).
```

4.2. Addition de vecteurs ternaires : add_vect3/3

C'est la même structure que add_vect/3 mais en utilisant add3 :

```
add_vect3([],[],[]).

add_vect3([H1|T1],[H2|T2],[R|T]) :-
   add3(H1,H2,R),
   add_vect3(T1,T2,T).
```

```
| ?- add_vect3([u,d],[d,u],R).
R = [o,o] ? ;
no
```

4.3. Produit vecteur – matrice ternaire : produit3/3

Le produit ternaire ajoute un troisième cas pour le coefficient d (qui est égal à 2) : multiplier une ligne par 2 revient à l'additionner avec elle-même (2 × Ligne = Ligne + Ligne), ce que l'on calcule avec add_vect3(Ligne, Ligne, Temp) :

```
produit3([], _, []).
produit3([o|T], [_|M], R) :-
   produit3(T, M, R).
produit3([u|T], [Ligne|M], R) :-
   produit3(T, M, R1),
   ( R1 = [] -> R = Ligne ; add_vect3(Ligne, R1, R) ).
produit3([d|T], [Ligne|M], R) :-
   produit3(T, M, R1),
   add_vect3(Ligne, Ligne, Temp),
   ( R1 = [] -> R = Temp ; add_vect3(Temp, R1, R) ).
```

4.4. Génération des vecteurs ternaires : vecteur3/2

```
vecteur3([],0).

vecteur3([H|T],N) :-
   N > 0,
   valeur_ternaire(H),
   N1 is N-1,
   vecteur3(T,N1).
```

Identique à vecteur/2 mais utilise valeur_ternaire/1 qui énumère o, u et d.

4.5. Comptage des éléments non nuls : count_ud/2

```
count_ud([],0).

count_ud([o|T],N) :-
   count_ud(T,N).

count_ud([u|T],N) :-
   count_ud(T,N1),
   N is N1 + 1.

count_ud([d|T],N) :-
   count_ud(T,N1),
   N is N1 + 1.
```

count_ud/2 ignore les o et compte les u et d. Cette clause permet d'imposer la contrainte sur le nombre d'éléments non nuls dans le vecteur solution**.**

4.6. Clause principale : solution_ternaire/4

```
solution_ternaire(M,V,Y,Nombre) :-
   length(M,N),
   vecteur3(V,N),
   count_ud(V,Nombre),
   produit3(V,M,Y).
```

Même architecture que solution_binaire/4 : on génère V, on filtre par count_ud avant de calculer le produit, et Y est le résultat attendu passé en paramètre.

Résultat :

```
| ?- solution_ternaire([[u,d],[d,u],[u,o],[o,u]],V,[o,o],3).

V = [o,u,u,d] ? ;

V = [o,d,d,u] ? ;

V = [u,o,d,u] ? ;

V = [d,o,u,d] ? ;

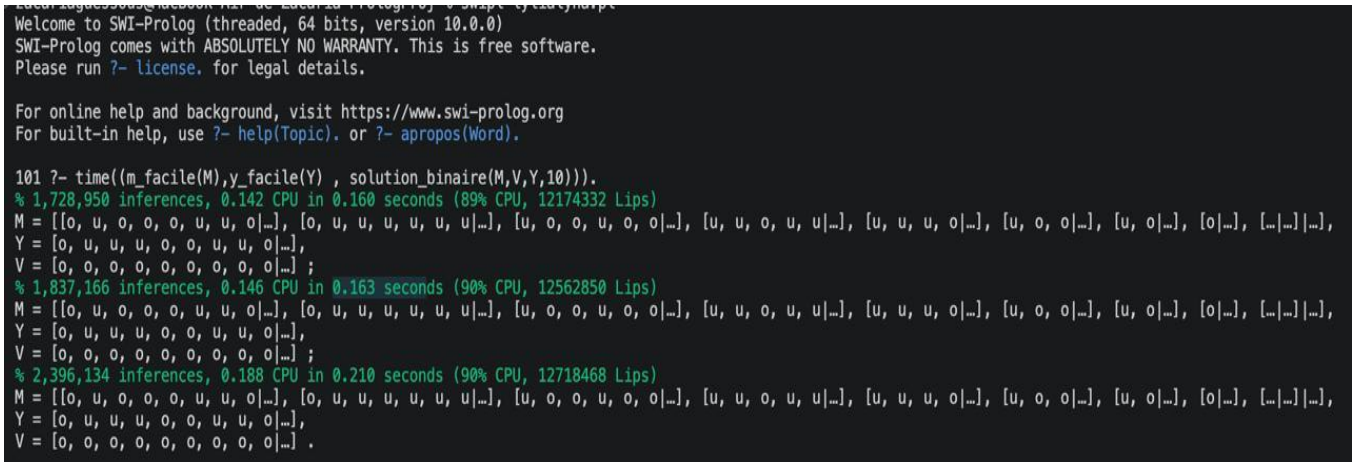
(16 ms) no
```

5. Les 3M

5.1. Binaire

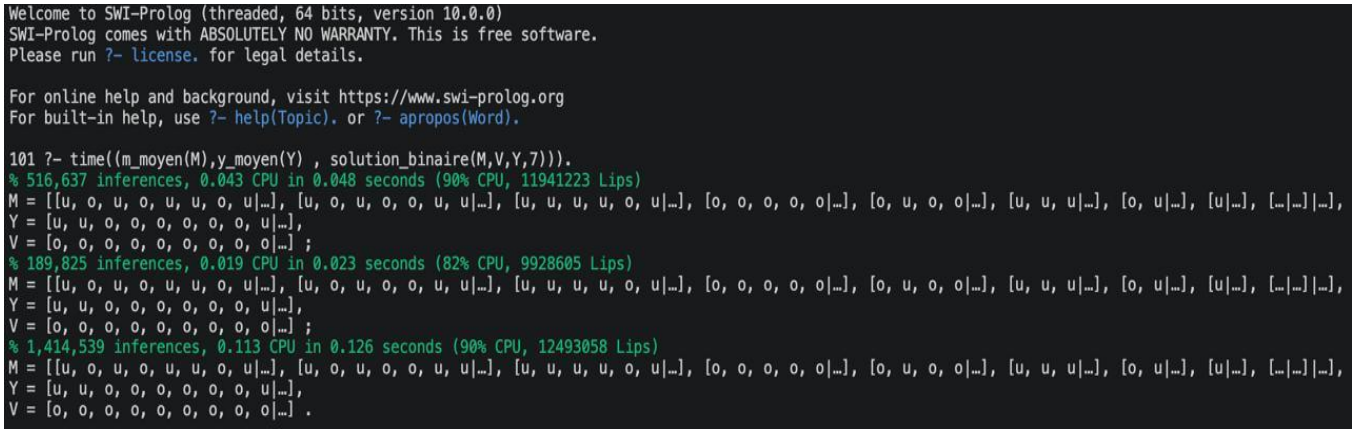
5.1.1. Facile

Requête : solution_binaire(m_facile, V, y_facile, 10). La capture ci-dessous montre un vecteur V contenant exactement 10 coefficients u tel que V × M_facile = Y_facile :

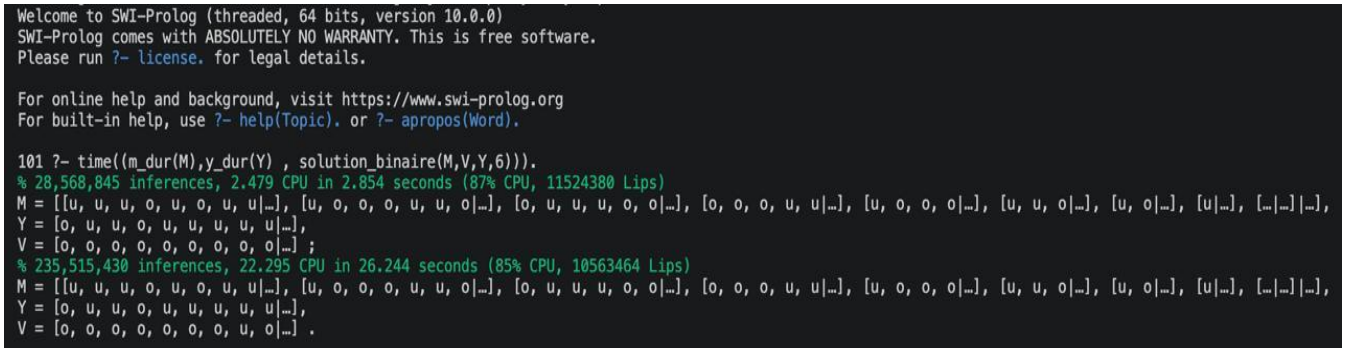


5.1.2. Moyen

Requête : solution_binaire(m_moyen, V, y_moyen, 7). La capture ci-dessous montre un vecteur V contenant exactement 7 coefficients u tel que V × M_moyen = Y_moyen :



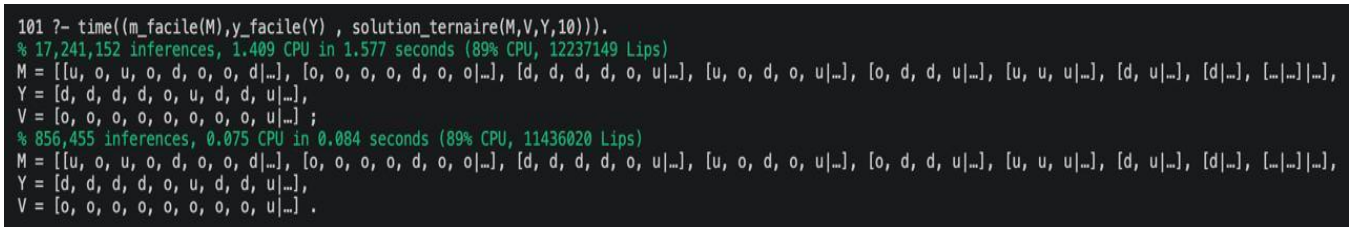
5.1.3. Dur



5.2. Ternaire

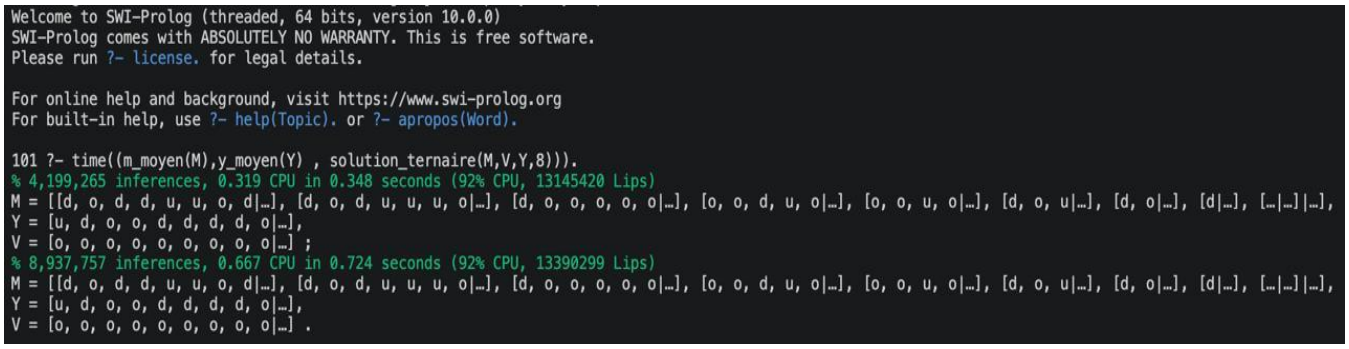
5.2.1. Facile

Requête : solution_ternaire(m_facile_t, V, y_facile_t, 10). La capture ci-dessous montre un vecteur V contenant exactement 10 coefficients non nuls (u ou d) tel que V × M_facile = Y_facile :



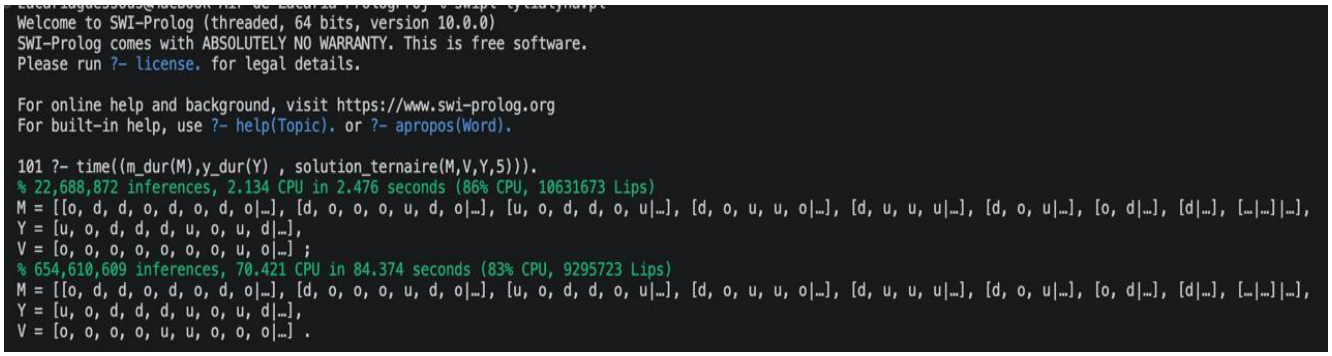
5.2.2. Moyen

Requête : solution_ternaire(m_moyen_t, V, y_moyen_t, 8). La capture ci-dessous montre un vecteur V contenant exactement 8 coefficients non nuls (u ou d) tel que V × M_moyen = Y_moyen :



5.2.3. Dur

Requête : solution_ternaire(m_dur_t, V, y_dur_t, 5). La capture ci-dessous montre un vecteur V contenant exactement 5 coefficients non nuls (u ou d) tel que V × M_dur = Y_dur. C'est le niveau le plus difficile : l'espace de recherche ternaire est immense (3^30 possibilités) :



Le programme génère tous les vecteurs binaires possibles de taille N. Or, le nombre de possibilités augmente très rapidement : 2^N en binaire et 3^N en ternaire. Pour les grandes matrices, le temps d'exécution devient donc important.

Optimisation principale : appliquer le filtre count_u (ou count_ud) avant le calcul du produit. Sans cette optimisation, Prolog calculait VxM pour tous les candidats, y compris ceux qui ne respectent pas la contrainte sur le nombre de u. Avec ce filtrage, seuls les vecteurs ayant exactement le bon nombre de u (ou de u/d) sont évalués.

Le tableau ci-dessous présente les temps d'exécution observés (en secondes) pour obtenir la première solution :

Niveau/temps(s)	Binaire	Ternaire
Facile	0.160 s	1.577 s
Moyen	0.048 s	0.348 s
Dur	2.854 s	84.374 s

6. Conclusion

Ce projet nous a permis de mettre en pratique les principes de la programmation logique en Prolog à travers la manipulation de vecteurs et de matrices en arithmétique binaire et ternaire. Nous avons utilisé la récursivité, les listes et les contraintes pour construire des programmes capables de résoudre des équations de la forme : V×M= Y avec des contraintes sur le nombre de coefficients non nuls du vecteur V.

Nous avons également appris à écrire un code générique capable de fonctionner avec des matrices de tailles quelconques, sans utiliser de dimensions fixées dans le programme. Enfin, nous avons amélioré les performances grâce à des optimisations permettant de réduire les calculs inutiles et d'obtenir de meilleurs temps d'exécution sur les grandes matrices.

Ce projet nous a ainsi permis de mieux comprendre le fonctionnement de Prolog, la programmation déclarative et la résolution de problèmes par contraintes.

7. Bibliographie

- SWI-Prolog documentation : https://www.swi-prolog.org/pldoc/doc_for?object=manual
- Supports de cours de Logique, L2 Informatique, Université de Limoges : https://community-sciences.unilim.fr/course/view.php?id=2508